

XEZIM

--module-timescale

Command-Line Extension Specification

Draft 0.1 | July 2026

Normative behavior for assigning timescales to modules that do not carry a protected, source-explicit time declaration.

Status: This document specifies a Xezim implementation extension. It is not part of IEEE Std 1800 SystemVerilog.

1 Scope

This extension defines the syntax, precedence, matching, diagnostics, and simulation semantics of the Xezim command-line option `--module-timescale`.

The option enables a user to assign one or more timescale values to selected module definitions, or to all eligible module definitions, without modifying the SystemVerilog source.

A source-explicit time declaration shall not be overridden by `--module-timescale`.

2 Normative terminology

Term	Definition
source-explicit time declaration	A module-local timeunit or timeprecision declaration, or a <code>`timescale</code> directive directly associated with that module definition.
directly associated <code>`timescale</code>	The most recent <code>`timescale</code> directive encountered after the end of the preceding design element and before the module keyword, after preprocessing.
inherited <code>`timescale</code>	A <code>`timescale</code> value still active from a directive associated with an earlier design element. It would apply under normal SystemVerilog compilation-unit rules, but it is not source-explicit for the current module under this extension.
eligible module	A module definition without a source-explicit time declaration.
named assignment	A <code>--module-timescale</code> assignment that lists one or more module identifiers.
global assignment	A <code>--module-timescale</code> assignment containing only a timescale value and no module list.

Design intent: This distinction allows module_c to receive a command-line timescale even when it would otherwise inherit module_b's active ``timescale`. A module may protect itself by declaring timeunit/timeprecision or by placing a new ``timescale` directive in its own preamble.

3 Command-line syntax

The implementation shall support both of the following forms:

```
--module-timescale <time_unit>/<time_precision>
--module-timescale <module_list>=<time_unit>/<time_precision>
```

The module list grammar shall be:

```
<module_list> ::= <module_identifier>
                | <module_identifier> , <module_list>
```

The option may appear more than once. Each occurrence shall define one assignment.

```
xezim \
--module-timescale 1ns/1ps \
--module-timescale module_c=10ns/1ns \
--module-timescale module_d,module_e=1ps/1ps \
design.sv
```

4 Timescale value requirements

The time unit and time precision shall use values accepted by the SystemVerilog implementation for timeunit and ``timescale` declarations.

- The time precision shall be equal to or finer than the time unit.
- An invalid unit, magnitude, or unit/precision relationship shall be an error.

- A command-line-assigned precision shall participate in determining the global simulation time precision.

```
# Legal examples
1ns/1ps
10ns/1ns
100ps/1ps

# Illegal example: precision is coarser than the unit
1ps/1ns
```

5 Source-explicit time protection

The `--module-timescale` option shall not override a source-explicit time declaration.

A module shall be protected when either of the following applies:

- The module definition contains a `timeunit` declaration, a `timeprecision` declaration, or a combined `timeunit/timeprecision` declaration.
- A ``timescale` directive is directly associated with the module definition as defined in Clause 2.

If either component is explicitly declared locally, the command-line assignment shall be ignored for the entire time-unit/time-precision pair. The remaining component shall be resolved using normal SystemVerilog rules.

5.1 Direct versus inherited ``timescale`

```
`timescale 1ns/1ps
module module_a;
endmodule

`timescale 10ns/1ns
module module_b;
endmodule

module module_c;
endmodule
```

Module	Source classification	Normal LRM result	<code>--module-timescale</code> eligibility
module_a	Directly associated <code>`timescale</code>	1ns/1ps	Not eligible; protected
module_b	Directly associated <code>`timescale</code>	10ns/1ns	Not eligible; protected
module_c	Inherited <code>`timescale</code>	10ns/1ns	Eligible

6 Global assignment semantics

The global form shall apply its value to every eligible module not selected by a named assignment.

```
--module-timescale 100ps/1ps
```

For the source in Clause 5.1, the effective timescales shall be:

Module	Effective timescale	Reason
module_a	1ns/1ps	Protected direct <code>`timescale</code>
module_b	10ns/1ns	Protected direct <code>`timescale</code>
module_c	100ps/1ps	Eligible; global assignment applied

7 Named assignment semantics

The named form shall apply its value only to matching eligible module definitions.

```
--module-timescale module_c,module_d=10ns/1ns
```

A named assignment shall have precedence over a global assignment. A named assignment shall not bypass source-explicit time protection.

```

xezim \
--module-timescale 1ns/1ps \
--module-timescale module_c=10ns/1ns \
--module-timescale module_d,module_e=1ps/1ps \
design.sv

```

Module class	Effective command-line assignment
module_c, if eligible	10ns/1ns
module_d and module_e, if eligible	1ps/1ps
Other eligible modules	1ns/1ps
Modules with source-explicit time	Source value; command-line assignment ignored

8 Precedence

The effective time unit and time precision of a module shall be selected using the following precedence, from highest to lowest:

1. Module-local timeunit and timeprecision declarations.
2. A directly associated `timescale directive.
3. A matching named --module-timescale assignment.
4. A global --module-timescale assignment.
5. An inherited `timescale directive.
6. A separate command-line default timescale, if supported.
7. The implementation-defined simulator default.

```

local timeunit/timeprecision
> direct `timescale
> named --module-timescale
> global --module-timescale
> inherited `timescale
> command-line default
> simulator default

```

9 Multiple assignments and conflicts

The option may be repeated to assign different timescales to different module groups.

```

--module-timescale module_a,module_b=1ns/1ps
--module-timescale module_c=10ns/1ns
--module-timescale module_d,module_e=1ps/1ps

```

The following conflict rules shall apply:

- Two different named values assigned to the same module shall be an error.
- Repeated identical named assignments may be accepted; the implementation may warn that they are redundant.
- A named assignment and a global assignment are not a conflict; the named assignment wins.
- A command-line assignment targeting a protected module shall be ignored, not treated as a conflict.

```

error: conflicting --module-timescale assignments for module 'module_c':
      1ns/1ps and 10ns/1ns

```

10 Matching and scope

- A module identifier shall match a module definition name after preprocessing and parsing.
- An assignment shall apply to the module definition, not to individual instances.
- All instances of the same matched module definition shall use the same effective timescale.
- This option shall apply only to modules. Interfaces, programs, packages, checkers, classes, and compilation-unit declarations are outside the scope of this option.
- A named assignment that matches no module should produce a warning. An implementation may offer a strict mode that promotes this warning to an error.

11 Diagnostics and reporting

When an assignment targets a protected module, the implementation should report that the assignment was ignored and identify the source of protection.

```
warning: --module-timescale for 'module_a' ignored;
        module has direct source `timescale 1ns/1ps
```

The implementation should provide a report showing the effective time unit, precision, and resolution source for every module.

```
xezim --report-timescales design.sv
module_a 1ns/1ps    direct source `timescale
module_b 10ns/1ns   direct source `timescale
module_c 100ps/1ps  named --module-timescale
module_d 1ns/1ps    global --module-timescale
module_e 10ns/1ns   inherited `timescale
```

12 Compilation units and preprocessing

Direct and inherited `timescale classification shall be performed after preprocessing and independently for each compilation unit.

- A `resetall directive clears the active inherited compiler-directive state according to normal implementation behavior.
- A `timescale directive introduced through an included file may be directly associated with the next module when it appears in that module's preamble after preprocessing.
- Source-file order and compilation-unit boundaries may change whether a module would otherwise inherit a `timescale; they shall not change whether a local declaration is protected.

13 Delay interpretation

After the effective time unit and time precision have been selected, delay values, time literals, rounding, and global scheduling shall follow normal SystemVerilog simulation semantics.

```
module module_c;
  initial begin
    #1;      // one effective time unit
    #0.125;  // rounded to the effective time precision
  end
endmodule
```

For example, when module_c is assigned 10ns/1ns, #1 represents 10 ns and #0.125 represents 1.25 ns before rounding to the 1 ns precision.

14 Complete example

```
`timescale 1ns/1ps
module module_a;
  initial #1 $display("A");
endmodule

`timescale 10ns/1ns
module module_b;
  initial #1 $display("B");
endmodule

module module_c;
  initial #1 $display("C");
endmodule

module module_d;
  timeunit 100ps / 1ps;
  initial #1 $display("D");
endmodule

`resetall
module module_e;
  initial #1 $display("E");
```

```

endmodule
xezim \
  --module-timescale 1ns/1ps \
  --module-timescale module_c=1ps/1ps \
  design.sv

```

Module	Effective timescale	#1 delay	Resolution source
module_a	1ns/1ps	1 ns	Direct source `timescale
module_b	10ns/1ns	10 ns	Direct source `timescale
module_c	1ps/1ps	1 ps	Named --module-timescale
module_d	100ps/1ps	100 ps	Local timeunit/timeprecision
module_e	1ns/1ps	1 ns	Global --module-timescale

15 Required implementation behavior

1. --module-timescale shall be repeatable and shall support both global and named forms.
2. Source-explicit time declarations shall not be overridden.
3. A named assignment shall override a global assignment for the same eligible module.
4. A named or global assignment may override an inherited `timescale.
5. Conflicting named assignments shall be errors.
6. Assignments shall apply to module definitions and all their instances.
7. Assigned precisions shall participate in global simulation precision.
8. The option shall be documented as a Xezim extension rather than an IEEE SystemVerilog feature.